

category: camera-integration

tags: [kztek-cameras, avalonia, skbitmap, snapshot, type-mismatch, use-after-free, accessviolationexception]

severity: critical

created: 2026-07-16

updated: 2026-07-16

project-origin: parking-v8-app-avalonia

`Camera.GetCurrentVideoFrameAsync()` trong Kztek.Cameras (Avalonia lib) trả `SKBitmap` **tham chiếu sống**** — TUYỆT ĐỐI KHÔNG dispose**

⚠ ⚠ CẬP NHẬT 2026-07-20 — LESSON NÀY ĐÃ LỖI THỜI, HỢP ĐỒNG ĐÃ ĐỔI NGƯỜC LẠI:

Sau khi sửa BUG-004 (SKBitmap là nguồn RAM leak — xem

[anvplayerservice-motionanalyzing-unsubscribed-bitmap-leak.md](#) cập nhật 4-5), toàn bộ display pipeline

đổi sang `Avalonia.Media.Imaging.WriteableBitmap`.

`GetCurrentVideoFrame()` / `GetCurrentVideoFrame2()`

giờ trả **BẢN SAO SKBitmap RIÊNG** (tạo on-demand từ WriteableBitmap nội bộ) — **PHẢI dispose** sau khi

dùng xong, NGƯỜC HẸN với quy tắc "tham chiếu sống, cấm dispose" mà lesson này mô tả bên dưới.

Rule "KHÔNG dispose" CHỈ còn áp dụng cho code CŨ trước 2026-07-20 (SKBitmap trực tiếp từ `_currentSKFrame`) — nếu thấy code hiện tại gọi các hàm này, **PHẢI** kiểm tra đã dùng `using/dispose`

đúng theo hợp đồng MỚI chưa (xem [docs/bugs/BUG-004-mainshell-freeze-after-login.md](#) mục "FIX ÁP DỤNG

(C2)"), KHÔNG áp dụng máy móc lesson cũ bên dưới.

⚠ CẬP NHẬT 2026-07-16 (critical): Phiên bản trước của lesson này khuyên "dispose thủ công" —

SAI, và chính khuyến nghị đó đã gây ra `System.AccessViolationException` production (crash tại `SKBitmapPool.Return()` khi đọc `bmp.Width` trên native memory đã bị free).

Đọc kỹ mục Root Cause + Giải pháp bên dưới trước khi áp dụng bất kỳ code nào gọi

`GetCurrentVideoFrameAsync()` / `GetCurrentVideoFrame()` / `GetCurrentVideoFrame2()`.

(Bối cảnh LỊCH SỬ — trước 2026-07-20, khi display pipeline còn dùng SKBitmap trực tiếp.)

Tình huống gặp phải

Đang implement `CaptureSnapshotBitmapAsync()` và `ProbeFirstFrameAsync()` trong `CameraView.axaml.cs` (ParkingV8.UI), cần chụp/kiểm tra frame hiện tại từ RTSP player đang chạy (using `Kztek.Cameras`; — class `Camera` từ external lib `Kztek.Cameras.Avalonia` trong `BaseLIB`, KHÁC với `ParkingV8.Camera.Models.Camera` nội bộ trả `Avalonia.Media.Imaging.Bitmap`).

Triệu chứng / Lỗi

```
System.AccessViolationException: Attempted to read or write protected memory.  
This is often an indication that other memory is corrupt.  
    at Kztek.Cameras.SKBitmapPool.Return(SKBitmap bmp) // dòng đọc  
    bmp.Width/Height/ColorType
```

Crash xảy ra ngẫu nhiên, thường sau khi vào trạng thái "Connecting" (camera vừa Start, `ProbeFirstFrameAsync()` chạy dò frame đầu tiên) hoặc sau khi bấm Snapshot.

Nguyên nhân gốc rễ (Root Cause)

`Camera.GetCurrentVideoFrameAsync()` (external lib) đi qua chuỗi gọi:

```
Camera.GetCurrentVideoFrame() → videoSourcePlayer.GetCurrentVideoFrame2()  
    → AnvPlayer.GetCurrentVideoFrame2() → Volatile.Read(ref _currentSKFrame)
```

`_currentSKFrame` là **field nội bộ**, dùng chung giữa: (1) decoder thread ghi frame mới qua `Interlocked.Exchange`, (2) UI render pipeline (`Render()/DrawImage`) đọc để vẽ, (3) sau khi bị swap ra, được `SKBitmapPool.Return()` — enqueue lại vào pool để tái dùng (hoặc `Dispose()` thật nếu pool đầy).

`GetCurrentVideoFrame2()` **KHÔNG clone** — trả thẳng con trỏ tới object đang sống này.

Contract này được ghi rõ trong doc comment của chính thư viện

(`frmViewCamera.axaml.cs` dòng 155-156: `/*đây là internal reference do SKBitmapPool quản lý. KHÔNG dispose frame sau khi đọc*/`), nhưng lesson bản cũ đã bỏ qua cảnh báo này và khuyên `finally { skBitmap.Dispose(); }`.

Hệ quả: caller dispose native SKBitmap trong khi nó **vẫn đang là `\_currentSKFrame` sống** — render pipeline hoặc `SKBitmapPool.Return()` (khi decoder swap frame tiếp theo) sau đó truy cập object đã bị free native memory → `AccessViolationException` (crash không deterministic, tùy thời điểm GC/thread nào chạm vào object trước).

Giải pháp

KHÔNG dispose bitmap trả về từ `GetCurrentVideoFrameAsync()`. Chỉ đọc dữ liệu (convert, đọc Width/Height) rồi bỏ qua — lifecycle thuộc `AnvPlayer/SKBitmapPool`.

```
public async Task<Bitmap?> CaptureSnapshotBitmapAsync()  
{  
    if (camera is null) { UpdateStatus(...); return null; }  
    try  
    {
```

```
var skBitmap = await camera.GetCurrentVideoFrameAsync().ConfigureAwait(true);
if (skBitmap is null) { UpdateStatus(...); return null; }

// KHÔNG dispose skBitmap — tham chiếu sống do AnvPlayer/SKBitmapPool quản lý.
var avaloniaBitmap = skBitmap.ToAvaloniaBitmapJpeg();
if (avaloniaBitmap is null) { UpdateStatus(...); return null; }

MarkFirstFrameReceived();
UpdateStatus("Live", $"Da chup snapshot luc {DateTime.Now:HH:mm:ss}.",
"Brush.CameraBadgeSuccess", false);
return avaloniaBitmap;
}
catch (Exception ex) { ...; return null; }
}
```

`ProbeFirstFrameAsync()` cũng sửa tương tự — bỏ `bitmap.Dispose()`; chỉ dùng `bitmap is not null` để biết đã có frame.

Bước thực hiện:

1. Tìm mọi nơi gọi `camera.GetCurrentVideoFrameAsync()` / `GetCurrentVideoFrame()` / `GetCurrentVideoFrame2()` trên object `Kztek.Cameras.Camera` (external lib).
2. Xoá mọi `.Dispose()` gọi trên kết quả trả về của các hàm này.
3. Nếu cần giữ dữ liệu lâu dài (không chỉ đọc ngay) → dùng `skBitmap.Copy()` trước, dispose bản copy tự tạo (an toàn vì đó là object riêng, không phải `_currentSKFrame`).

Áp dụng lại (How to reuse)

- Khi thấy code dùng `using Kztek.Cameras;` (external lib) gọi bất kỳ hàm `GetCurrentVideoFrame*`() nào → mặc định coi kết quả là **tham chiếu sống, KHÔNG dispose**, trừ khi đọc rõ trong source lib rằng hàm đó trả bản clone (`.Copy()`).
Kiểm tra bằng Grep `Volatile.Read / _currentSKFrame` trong `AnvPlayer.cs` nếu còn nghi ngờ.
- Khi cần một bản bitmap sở hữu riêng (để giữ lâu, để dispose an toàn) → gọi `skBitmap.Copy()` rồi làm việc trên bản copy, **KHÔNG** bao giờ dispose bản gốc.
- `AccessViolationException` bất kỳ đâu liên quan `SKBitmapPool/SKBitmap` trong project này → nghi ngờ đầu tiên là "ai đó dispose frame dùng chung" — Grep toàn bộ `.Dispose()` sau các cuộc gọi `GetCurrentVideoFrame*`() .

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `SKBitmap.Dispose()` **không throw** ngay lập tức khi gọi nhầm — exception xuất hiện SAU đó, ở một chỗ hoàn toàn khác (`SKBitmapPool.Return()`, hoặc trong `Render()`), khiến việc debug dễ đi sai hướng nếu chỉ nhìn stack trace của exception.
→ Luôn kiểm tra ngược lên các nơi gọi `GetCurrentVideoFrame*`() + `.Dispose()` trước.
- ⚠ `ParkingV8.Camera/Models/Camera.cs` là class **khác** — namespace `Kztek.Cameras.Models`, trả `Avalonia.Media.Imaging.Bitmap` thật sự sở hữu riêng, **được phép** dispose bình thường.

Đừng áp dụng quy tắc "không dispose" nhầm sang class này.

- ⚠ `ToAvaloniaBitmapJpeg()` có thể trả `null` (nếu encode lỗi) — phải null-check trước khi return.
- ⚠ Trong `AnvPlayer.ConsumerDisplayFrameAsync`, catch-block gốc có bug tương tự (double-ownership):

nếu exception xảy ra SAU khi đã `Interlocked.Exchange(ref _currentSKFrame, bmp)` (frame mới đã là current), catch cũ vẫn `SKBitmapPool.Return(bmp)` — trả nhầm frame đang sống về pool. Nếu sửa file `AnvPlayer.cs` (external lib), nhớ track cò `swapped` để chỉ Return khi CHƯA swap.

Tham chiếu

- `src/ParkingV8.UI/Controls/Cs/CameraView.axaml.cs` — file fix (bỏ `.Dispose()` ở 2 chỗ)
- `src/ParkingV8.UI/Extensions/SkiaAvaloniaAdapter.cs` — `ToAvaloniaBitmapJpeg()` extension
- `E:\KZTEK\Code_Git\1.Window\0.BaseLIB\Kztek.Camera\...\Kztek.Cameras.Avalonia\Camera.cs` — external lib, `GetCurrentVideoFrame()/GetCurrentVideoFrameAsync()`
- `E:\KZTEK\Code_Git\1.Window\0.BaseLIB\Kztek.Camera\...\Players\FFMPEG\UserControls\AnvPlayer.cs` — `_currentSKFrame, GetCurrentVideoFrame2(), ConsumerDisplayFrameAsync` (nơi swap + Return)
- `E:\KZTEK\Code_Git\1.Window\0.BaseLIB\Kztek.Camera\...\Players\FFMPEG\UserControls\SKBitmapPool.cs` — nơi crash biểu hiện ra (`Return()` đọc `bmp.Width` trên object đã free)
- Project liên quan: `parking-v8-app-avalonia`